

华东师范大学数据科学与工程学院实验报告

课程名称: AI 基础	年级: 2024	上机实践时间: 2026.03.27
指导教师: 杨彬	姓名: 孙育泉	
上机实践名称: Project 1	学号: 10234900421	

I 实验任务

I.1 人狼羊菜

一个农民要带着一只狼、一只羊、一颗白菜过河。

- (1) 人不在的时候，狼会吃羊、羊会吃草
- (2) 人每次只能带一样东西过河。

有 8 种可能的动作: $F \triangleright$, $F \triangleleft$, $FC \triangleright$, $FC \triangleleft$, $FG \triangleright$, $FG \triangleleft$, $FW \triangleright$, $FW \triangleleft$, 其中符号表示为: F (Farmer), C (Cabbage), G (Goat), W (Wolf), $\triangleright$ 和 $\triangleleft$ 分别表示向右和向左过河。

要求:

- (1) 画出状态空间图
- (2) 基于你的状态空间图，分别用 BFS 和 DFS 算法画出搜索路径
- (3) 挑选 BFS 或 DFS 算法，写出搜索到最终状态的流程
 - BFS 展示每一步的队列
 - DFS 展示每一步的递归栈

I.2 图最短路问题

- (1) 在边权为 1 的有向图上，使用 BFS 算法求 SSSP.
- (2) 在正边权的有向图中，使用朴素 Dijkstra 算法求 SSSP.
- (3) 在正边权的有向图中，使用堆优化 Dijkstra 算法求 SSSP.

I.3 八数码问题

- (1) 使用 DFS 判断八数码问题的可解性.
- (2) 使用 BFS 求解八数码问题.
- (3) 使用 Dijkstra 算法求解八数码问题.
- (4) 使用 A* 算法求解八数码问题.

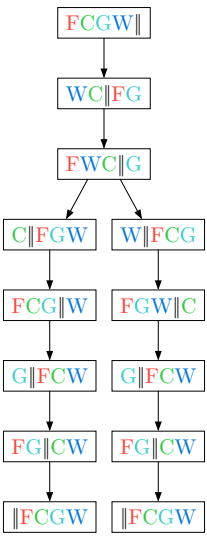
II 使用环境

编程语言: Python 3

III 实验过程

III.1 人狼羊菜

搜索树如下：



分别使用 BFS 和 DFS 算法搜索到最终状态的路径如下（使用编号来记录访问的状态顺序）：



本次选择使用 BFS 算法.首先要考虑如何记录状态, 由于每个角色有互斥关系, 并且分为河的两侧, 于是考虑使用状态压缩来表示一侧的状态, 然后对于完整的两侧状态使用元组即可, 也就是 tuple[int, int].表示如下:

py

```

1 FARMER, CABBAGE, GOAT, WOLF = 1, 2, 4, 8
2 ROLES = "FCGW"
3
4 initial_state = FARMER | CABBAGE | GOAT | WOLF, 0
5 final_state = 0, FARMER | CABBAGE | GOAT | WOLF
    
```

接下来, 我们考虑如何判断一个状态是否合法. 根据题目要求, 农民不在的时候, 狼会吃羊、羊会吃草, 所以我们需要判断在某一侧如果没有农民, 那么是否存在狼和羊或者羊和草同时存在的情况, 如果存在则说明这个状态不合法.

py

```

1 def valid(state: tuple[int, int]) → bool:
2     def check(side: int) → bool:
3         if (side & CABBAGE) and (side & GOAT):
4             return False
5         if (side & GOAT) and (side & WOLF):
6             return False
7         return True
8
9     l, r = state
10    return check(r) if l & FARMER else check(l)

```

然后, 为了方便将状态转换成可读的字符串, 我们可以写一个函数来将状态转换成字符串表示:

py

```

1 def state_to_str(state: tuple[int, int]) → str:
2     def foo(side: int):
3         return "".join(ROLES[i] for i in range(4) if (side & (1 << i)) ≠ 0)
4
5     l, r = state
6     return f"{foo(l):<4} | {foo(r)}"

```

对于 BFS 部分, 因为要输出不同的方案, 因此我们考虑对于队列的每一项都记录一个到该状态的路径, 同时使用 vis 字典来记录到当前状态的最短路径长度, 保证最优解. 然后, 使用 step 计数每次层级的扩展, 用于输出每层时的队列状态. 在进行状态转移的时候, 只需要使用异或操作来翻转对应角色的状态, 然后检测是否是合法状态即可. 最后, 输出所有的方案.

py

```

1 def bfs():
2     # path of states
3     q = deque([[initial_state]])
4
5     vis = {initial_state: 0}
6
7     solutions = []
8     step = 1
9
10    while q:
11        print(f"=== step {step} queue ===")
12        for path in q:
13            print(" " + state_to_str(path[-1]))
14        print()
15
16        level_size = len(q)
17        for _ in range(level_size):
18            path = q.popleft()
19            current_state = path[-1]
20            current_depth = len(path) - 1
21
22            if current_state == final_state:
23                solutions.append(path)
24                continue
25
26            l, r = current_state

```

```

27         for i in range(4):
28             next_state = None
29             if (l & FARMER) and (l & (1 << i)):
30                 next_state = (l ^ (FARMER | (1 << i)), r ^ (FARMER | (1 << i)))
31             elif (r & FARMER) and (r & (1 << i)):
32                 next_state = (l ^ (FARMER | (1 << i)), r ^ (FARMER | (1 << i)))
33
34             if next_state is not None and valid(next_state):
35                 if next_state not in vis:
36                     vis[next_state] = current_depth + 1
37                     q.append(path + [next_state])
38                 elif vis[next_state] == current_depth + 1:
39                     q.append(path + [next_state])
40         if solutions:
41             break
42
43     step += 1
44
45     print("-" * 30)
46     print(f"find {len(solutions)} best solutions:\n")
47     for idx, sol in enumerate(solutions, 1):
48         print(f"solution {idx}")
49         for s in sol:
50             print(" " + state_to_str(s))
51     print()

```

最后，整个程序的输出如下：

txt

```

1  === step 1 queue ===
2  FCGW |
3
4  === step 2 queue ===
5  CW   | FG
6
7  === step 3 queue ===
8  FCW  | G
9
10 === step 4 queue ===
11 W    | FCG
12 C    | FGW
13
14 === step 5 queue ===
15 FGW  | C
16 FCG  | W
17
18 === step 6 queue ===
19 G    | FCW
20 G    | FCW
21
22 === step 7 queue ===
23 FG   | CW
24 FG   | CW
25
26 === step 8 queue ===
27     | FCGW

```

```

28 |         | FCGW
29 |
30 |-----
31 find 2 best solutions:
32
33 solution 1
34 FCGW |
35 CW   | FG
36 FCW  | G
37 W    | FCG
38 FGW  | C
39 G    | FCW
40 FG   | CW
41     | FCGW
42
43 solution 2
44 FCGW |
45 CW   | FG
46 FCW  | G
47 C    | FGW
48 FCG  | W
49 G    | FCW
50 FG   | CW
51     | FCGW

```

III.2 图最短路问题

为了简化代码的复用，这里统一一些变量和记号的定义：

记号	定义
n	图中节点的数量
m	图中边的数量
g[][]	图的邻接表表示
d[]	距离数组
q[]	队列 / 优先队列
flg[]	访问标记数组

表 1: SSSP 记号

III.2.1 BFS

因为边权都是 1，所以扩展的步数就是最短路长度，因此可以直接使用 BFS 来求解 SSSP 问题。

```

1 def bfs():
2     global d
3     q, d[1] = deque([1]), 0
4     while len(q):
5         u = q.popleft()
6         for v in g[u]:
7             if d[v] > d[u] + 1:
8                 d[v] = d[u] + 1
9                 q.append(v)

```

III.2.2 朴素 Dijkstra

因为没有负权边，所以可以利用贪心：每次选择当前距离最小的没有扩展过的节点进行扩展，直到所有节点都被访问过或者无法访问为止。

py

```

1 def dijkstra():
2     global d, flg
3     d[1] = 0
4     for _ in range(n):
5         u = -1
6         for i in range(1, n + 1):
7             if not flg[i] and (u == -1 or d[i] < d[u]):
8                 u = i
9         if u == -1:
10            break
11        flg[u] = True
12        for v, w in g[u]:
13            if not flg[v] and d[v] > d[u] + w:
14                d[v] = d[u] + w

```

III.2.3 堆优化 Dijkstra

可以使用小根堆来优化 Dijkstra 算法中寻找当前距离最小的节点的过程，从而将算法的时间复杂度从 $O(n^2)$ 降低到 $O(m \log n)$ 。

py

```

1 def dijkstra():
2     global d
3     d[1], pq = 0, [(0, 1)]
4
5     while pq:
6         dis, u = heappop(pq)
7
8         if dis > d[u]:
9             continue
10
11        for v, w in g[u]:
12            if d[v] > d[u] + w:
13                d[v] = d[u] + w
14                heappush(pq, (d[v], v))

```

III.3 八数码问题

同样，这里声明一些记号：

记号	定义
a	初始状态
tar	目标状态
d[]	距离哈希表
valid()	判断状态是否合法的函数
flg	访问标记

表 2: 八数码问题记号

III.3.1 DFS

因为 DFS 可能会重复搜索，所以用一个集合来记录已经访问过的状态，避免重复搜索。同时，因为 python 默认的递归深度较小，可以手动维护栈，但这里直接将递归深度限制调大。

py

```

1 import sys
2 sys.setrecursionlimit(200000)
3
4 # .....
5
6 def dfs(s: str) → bool:
7     if s == tar:
8         return True
9
10    x = s.index("x")
11
12    for dx, dy in ((-1, 0), (1, 0), (0, -1), (0, 1)):
13        nx, ny = x // 3 + dx, x % 3 + dy
14        if valid(nx, ny):
15            t = list(s)
16            t[x], t[nx * 3 + ny] = t[nx * 3 + ny], t[x]
17            t = "".join(t)
18            if t not in flg:
19                flg.add(t)
20                if dfs(t):
21                    return True
22
23    return False
24
25 # .....

```

III.3.2 BFS

维护一个队列，每次扩展一个状态时，将其所有合法的下一步状态加入队列，并记录它们的距离。直到找到目标状态或者队列为空为止。

py

```

1 def bfs() → bool:
2     q = deque()
3     q.append((a, a.index("x"), 0))
4     d[a] = 0
5     while len(q) ≠ 0:
6         s, x, step = q.popleft()
7
8         if s == tar:
9             return True
10
11        r, c = x // 3, x % 3
12        for dx, dy in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
13            nr, nc = r + dx, c + dy
14
15            if valid(nr, nc):
16                nx = nr * 3 + nc
17
18                ns = list(s)
19                ns[x], ns[nx] = ns[nx], ns[x]
20                ns = "".join(ns)
21

```

```

22         if ns not in d:
23             d[ns] = step + 1
24             q.append((ns, nx, step + 1))
25     return False

```

III.3.3 Dijkstra

将状态空间看成一个图，每个状态是一个节点，每条合法的状态转移是一条边，边权为 1，所以问题就转换成 SSSP 问题。

py

```

1 def dijkstra() → bool:
2     q = []
3     heappush(q, (0, a, a.index("x")))
4     d[a] = 0
5     while len(q) ≠ 0:
6         step, s, x = heappop(q)
7
8         if s == tar:
9             return True
10
11         r, c = x // 3, x % 3
12         for dx, dy in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
13             nr, nc = r + dx, c + dy
14
15             if valid(nr, nc):
16                 nx = nr * 3 + nc
17
18                 ns = list(s)
19                 ns[x], ns[nx] = ns[nx], ns[x]
20                 ns = "".join(ns)
21
22                 if ns not in d or step + 1 < d[ns]:
23                     d[ns] = step + 1
24                     heappush(q, (step + 1, ns, nx))
25     return False

```

III.3.4 A*

启发式搜索，启发函数可以使用曼哈顿距离，即每个数字与其目标位置的行距和列距之和，保证不会高估实际距离，从而保证算法的正确性。

py

```

1 from heapq import heappush, heappop
2
3 d:dict[str, tuple[int, str]] = dict()
4 MOVES = {"u": (-1, 0), "d": (1, 0), "l": (0, -1), "r": (0, 1)}
5
6
7 def check(s: str) → bool:
8     cnt = 0
9     for i in range(9):
10         for j in range(i + 1, 9):
11             if s[i] ≠ "x" and s[j] ≠ "x" and s[i] > s[j]:
12                 cnt += 1
13     return cnt % 2 == 0
14
15
16 def g(s: str) → int:

```



```

17     return d[s][0]
18
19
20 def h(s: str) → int:
21     ret = 0
22     for i, c in enumerate(s):
23         if c != "x":
24             ti = int(c) - 1
25             ret += abs(ti // 3 - i // 3) + abs(ti % 3 - i % 3)
26     return ret
27
28
29 def f(s: str) → int:
30     return g(s) + h(s)
31
32
33 def astar():
34     q = []
35     d[a] = (0, "")
36     heappush(q, (f(a), a))
37     while q:
38         _, s = heappop(q)
39         if s == tar:
40             return
41         i = s.index("x")
42         x, y = i // 3, i % 3
43         for op, (dx, dy) in MOVES.items():
44             nx, ny = x + dx, y + dy
45             if valid(nx, ny):
46                 ns = list(s)
47                 ns[i], ns[nx * 3 + ny] = ns[nx * 3 + ny], ns[i]
48                 ns = "".join(ns)
49                 if ns not in d or d[ns][0] > d[s][0] + 1:
50                     d[ns] = (d[s][0] + 1, d[s][1] + op)
51                 heappush(q, (f(ns), ns))

```



Idea

这里判断是否有解利用了八数码问题的一个结论：有解 $\Leftrightarrow$ 初始状态和目标状态的逆序对数量奇偶性相同。

IV 总结

本次实验总结如下：

- (1) **SSSP 算法选择**：单位边权图直接使用 BFS 求解。正权图使用 Dijkstra，结合小根堆优化可将复杂度从 $O(n^2)$ 降至 $O(m \log n)$ 。
- (2) **状态空间搜索**：八数码问题本质可转化为图论中的 SSSP。DFS、BFS 与朴素 Dijkstra 作为盲目搜索，存在状态重复、易达递归极限或可能导致资源消耗过高的问题。
- (3) **启发式优化**：采用曼哈顿距离作为启发函数的 A* 算法，是控制状态搜索规模的有效手段。它保证了不被高估的实际距离，从而得到最优解。
- (4) **理论剪枝**：通过逆序对奇偶性判定八数码的可解性。可以避免无意义算力浪费。

i Info

完整代码见压缩包其余代码文件.